🛠️ Example: The "SkyCast" Backlog & Sprint Planning

Writing your backlog in Natural Language (NL) using the User Story format is critical. It prevents "Developer Tunnel Vision" where we focus only on the code and forget the human being using the product. Using a "Weather App" is to demonstrate these concepts because everyone understands the basic functionality, allowing students to focus purely on the Agile workflow rather than complex business logic.

Here is the detailed practice and solution for the "SkyCast" Weather Application.

Section 1: "SkyCast" Weather App User Story

Practice Objectives

- Translate high-level needs into structured User Stories.
- Practice Relative Estimation using Story Points.
- Master the art of Incremental Delivery across three Sprints.

Step-by-Step Instructions for Students

1. **Decomposition:** Break the app idea into 9 distinct functional requirements.
2. **Formatting:** Write each as a User Story: "As a [user], I want to [action], so that [value]."
3. **Estimation:** Assign Story Points (1, 2, 3, 5, 8) based on technical risk and effort.
4. **Prioritization:** Identify the "Critical Path"—the features without which the app cannot function.
5. **Sprint Assignment:** Organize the items into 3 Sprints, each with a specific Sprint Goal.

Here is the complete list of User Stories for the SkyCast project:

📍 Scenario: "SkyCast" Weather App

The Goal: To provide users with a reliable, intuitive, and data-driven way to plan their day around the weather.

1. The Core Infrastructure (Sprint 1)

- **PB1 (The Data Source):** As a user, I want the app to fetch data from a professional weather service so that I can rely on the accuracy of the temperatures and conditions displayed.

- **PB2 (Location Awareness):** As a commuter, I want the app to automatically detect my current GPS location so that I can see the immediate weather outlook without typing anything.
- **PB3 (Manual Discovery):** As a traveler, I want to search for weather by city name or zip code so that I can check conditions for places I plan to visit.

2. Utility & Customization (Sprint 2)

- **PB4 (Extended Planning):** As an outdoor enthusiast, I want to view a 7-day daily forecast with high and low temperatures so that I can plan my hiking or sporting events for the coming week.
- **PB5 (Regional Flexibility):** As an international student, I want to toggle between Celsius and Fahrenheit so that I can understand the temperature in the scale I am most familiar with.
- **PB6 (Efficiency):** As a frequent traveler, I want to save a list of "Favorite" cities so that I can switch between my home, work, and vacation spots with a single tap.

3. Advanced Features & Retention (Sprint 3)

- **PB7 (Safety First):** As a resident in a storm-prone area, I want to receive real-time push notifications for severe weather alerts so that I can take cover or protect my property before a storm hits.
- **PB8 (Visual Data):** As a hobbyist meteorologist, I want to view an interactive radar map showing rain and cloud movement so that I can track the exact path of approaching weather systems.
- **PB9 (Continuity):** As a multi-device user, I want my saved cities and settings to sync to the cloud so that I have the same personalized experience on both my phone and tablet.

Students, notice three things about this list that you should replicate in your own work:

1. Avoided Technical Jargon: We didn't say "As a user, I want a JSON API call." We said "I want to rely on the accuracy." The tech is the tool, but the trust is the value.
2. Clear Personas: We used different roles—"Commuter," "Outdoor Enthusiast," "International Student." This helps you realize that different users have different needs for the same app.
3. The "So That" Clause: Every story explains the Value Proposition. If you can't find a good "so that" for a feature, you should probably delete that feature from your backlog.

Section 2: 📍 "SkyCast" Backlog & Sprint Planning

The Goal: Build a responsive weather application that provides accurate, real-time data and personalized alerts for users globally.

---

1. The Product Backlog

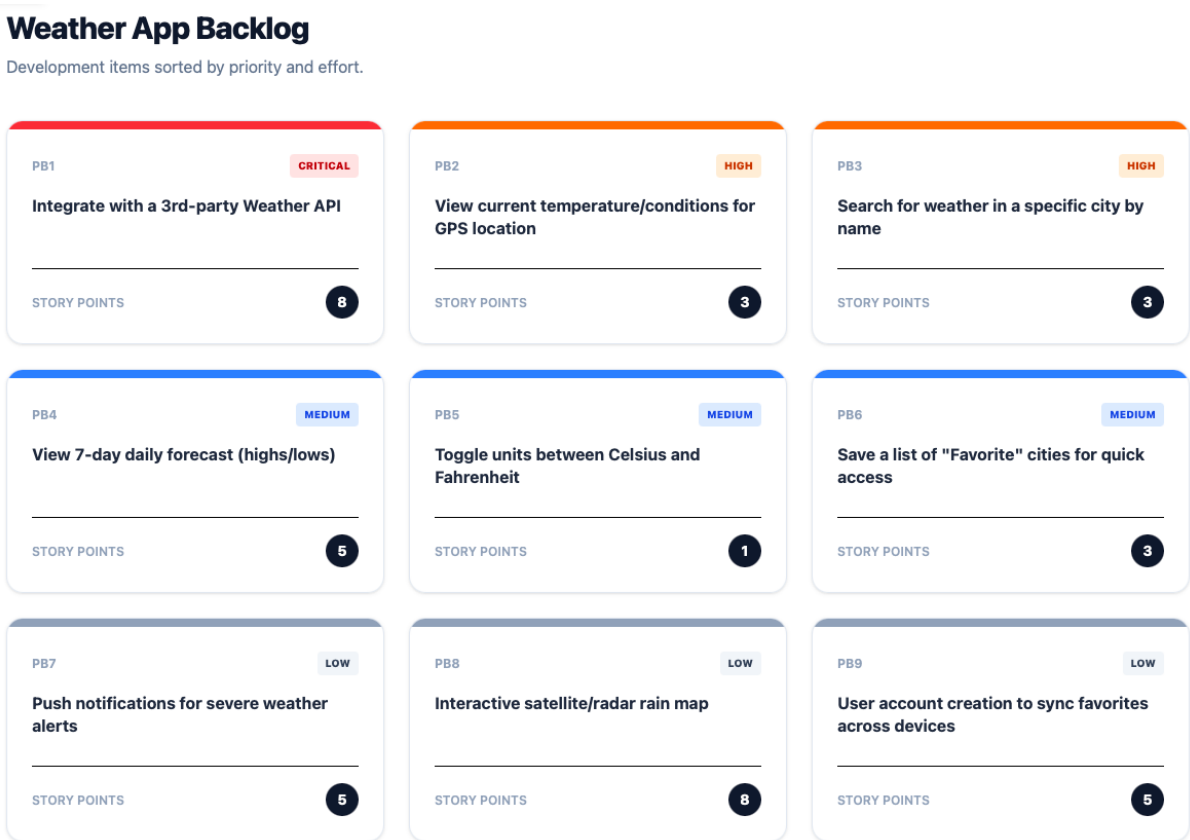
User Story	Product Backlog
PB1 (The Data Source): As a user, I want the app to fetch data from a professional weather service so that I can rely on the accuracy of the temperatures and conditions displayed.	Integrate with a 3rd-party Weather API (e.g., OpenWeather)
PB2 (Location Awareness): As a commuter, I want the app to automatically detect my current GPS location so that I can see the immediate weather outlook without typing anything.	View current temperature/conditions for current GPS location
PB3 (Manual Discovery): As a traveler, I want to search for weather by city name or zip code so that I can check conditions for places I plan to visit.	Search for weather in a specific city by name
PB4 (Extended Planning): As an outdoor enthusiast, I want to view a 7-day daily forecast with high and low temperatures so that I can plan my hiking or sporting events for the coming week.	View 7-day daily forecast (highs/lows)
PB5 (Regional Flexibility): As an international student, I want to toggle between Celsius and Fahrenheit so that I can understand the temperature in the scale I am most familiar with.	Toggle units between Celsius and Fahrenheit
PB6 (Efficiency): As a frequent traveler, I want to save a list of "Favorite" cities so that I can switch between my home, work, and vacation spots with a single tap.	Save a list of "Favorite" cities for quick access
PB7 (Safety First): As a resident in a storm-prone area, I want to receive real-time push notifications for severe weather alerts so that I can take cover or protect my property before a storm hits.	Push notifications for severe weather alerts
PB8 (Visual Data): As a hobbyist meteorologist, I want to view an interactive radar map showing rain and cloud movement so that I can track the exact path of approaching weather systems.	Interactive satellite/radar rain map
PB9 (Continuity): As a multi-device user, I want my saved cities and settings to sync to the cloud so that I have the same personalized experience on both my phone and tablet.	User account creation to sync favorites across devices

ID	Product Backlog	Story Points	Priority
PB1	Integrate with a 3rd-party Weather API (e.g., OpenWeather)	8	Critical
PB2	View current temperature/conditions for current GPS location	3	High
PB3	Search for weather in a specific city by name	3	High
PB4	View 7-day daily forecast (highs/lows)	5	Medium
PB5	Toggle units between Celsius and Fahrenheit	1	Medium
PB6	Save a list of "Favorite" cities for quick access	3	Medium
PB7	Push notifications for severe weather alerts	5	Low
PB8	Interactive satellite/radar rain map	8	Low
PB9	User account creation to sync favorites across devices	5	Low

---

Card Vew: Weather App Backlog

These card views are ready to be add to each sprint, Scrum board, or Kanban board.



---

2. Sprint Arrangement

Sprint	Sprint Goal	Included Items	Total Points
Sprint 1	The Core MVP	PB1 (API Integration), PB2 (GPS Current Temp), PB3 (City Search)	14
Sprint 2	Enhanced Utility	PB4 (7-Day Forecast), PB5 (Unit Toggle), PB6 (Favorite Cities)	9
Sprint 3	Advanced Features	PB7 (Alerts), PB8 (Radar Map), PB9 (Account Sync)	18

Discussion:

- **The "API First" Strategy:** In Sprint 1, PB1 (API Integration) is the "Heavy Lifter." Without the data source, PB2 and PB3 are impossible to build. (Technical Dependencies).
- **Defining the MVP:** "Could we ship the app after Sprint 1?" The answer is Yes. It's basic, but a user can check the weather. That is the essence of Agile—delivering value early.
- **Managing Complexity:** Notice how the Radar Map (PB8) is in Sprint 3. It's high-effort (8 points) but lower priority than knowing the basic temperature. We save high-complexity/lower-value items for later to ensure the core product is stable first.
- **Velocity Balance:** Sprint 2 has fewer points (9) than Sprint 1 (14) or Sprint 3 (18). To explain that this, "breathing room" is often necessary in the middle of a project to handle bug fixes or refactoring from the first release.